

**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII
al REPUBLICII MOLDOVA**

UNIVERSITATEA TEHNICĂ A MOLDOVEI

**FACULTATEA CALCULATOARE, INFORMATICĂ
ȘI MICROELECTRONICĂ**

Departamentul Informatică și Ingineria Sistemelor

RAPORT

Aplicatii Client-Server

**Tema: Elaborarea unei aplicatii Client-Server: pentru elaborarea jocului de
"Șah"**

Executant: / Zaharenco Mihail CR-231 /

A verificat: /Rotaru L. /

Chișinău 2026

CUPRINS

INTRODUCERE	2
1. ANALIZA PIETII JOCURILOR INTELECTUALE	3
1.1. Analiza jocurilor intelectuale	3
1.1.1. Analiza Aplicației Chess.com	3
1.1.2. Analiza Aplicației Lichess.org	4
1.1.3. Analiza Aplicației World Chess	5
1.2. Analiza caracteristicilor aplicațiilor	6
1.3. Justificarea alegerii caracteristicile de baza a aplicației	6
1.4. Caracteristicile aplicației propuse – „Manager de partide de șah”	7
2. ANALIZA TEHNOLOGIILOR UTILIZATE ÎN APLICAȚIE	
2.1 Limbajul de programare Python	8
2.2 Framework-ul FastAPI	9
2.3 Baza de date SQLite	9
2.4 ORM-ul SQLAlchemy	9
2.5 Framework-ul React	10
2.6 Biblioteca react-chessboard	10
2.7 Biblioteca chess.js	11
2.8 WebSocket	11
2.9 Biblioteca Axios	11
3. CREAREA ALGORITMULUI ȘI A PROGRAMULUI	13
3.1 Descrierea schemei de funcționare a programului	13
3.2 Crearea algoritmul programului	14
3.3 Descrierea programului	15
3.3.1 Partea Client	16
3.3.2 Partea Server	18
3.3.3 Baze de Date	19
Concluzie	21
BIBLIOGRAFIA	22

INTRODUCERE

În prezent, jocul de șah a cunoscut o evoluție semnificativă datorită progreselor tehnologice și a extinderii mediului digital. De la o activitate tradițională desfășurată exclusiv pe o tablă fizică, șahul s-a transformat într-un fenomen global accesibil prin intermediul platformelor online. Astfel, utilizatorii nu mai sunt limitați de spațiu sau timp, ci pot juca partide cu adversari din orice colț al lumii, la orice oră, prin intermediul aplicațiilor web sau mobile. Această transformare a contribuit semnificativ la creșterea popularității jocului, atât în rândul amatorilor, cât și al profesioniștilor.

Platformele moderne de șah online, precum Chess.com, Lichess.org sau Chess24, oferă mult mai mult decât posibilitatea de a juca o partidă. Acestea integrează funcționalități avansate, cum ar fi analiza automată a mutărilor cu ajutorul motoarelor de inteligență artificială (precum Stockfish), generarea de statistici privind performanța jucătorului, salvarea automată a partidelor și posibilitatea de revizuire pas cu pas a fiecărei mișcări. În plus, ele oferă opțiuni pentru participarea la turnee internaționale, jocuri de antrenament, lecții interactive și provocări zilnice, creând o experiență completă pentru utilizatori.

Din punct de vedere tehnic, majoritatea acestor aplicații se bazează pe o arhitectură client–server, care permite o comunicare eficientă și rapidă între jucători și sistem. Partea de client (interfața utilizatorului) este responsabilă de afișarea grafică a tablei, preluarea comenzilor și trimiterea acestora către server. Partea de server gestionează logica principală a aplicației — validarea mutărilor, actualizarea bazei de date, sincronizarea în timp real între participanți și menținerea istoricului fiecărei partide. Această structură asigură atât stabilitate, cât și scalabilitate, fiind esențială pentru platformele cu mii de utilizatori activi simultan. Funcționalitățile cheie ale acestor aplicații includ, în mod obișnuit: crearea și autentificarea conturilor, salvarea automată a partidelor, sistemele de rating bazate pe algoritmul Elo, analiza mutărilor cu suport AI, generarea de rapoarte statistice și organizarea de turnee online. În plus, multe platforme moderne includ chat-uri integrate, liste de prieteni, clasificări globale și sisteme de recompense, contribuind astfel la crearea unei comunități interactive și dinamice.

Scopul acestui capitol este de a analiza principalele aplicații de tip client–server dedicate jocului de șah, pentru a identifica funcționalitățile esențiale, avantajele arhitecturale și practicile recomandate în dezvoltarea unui astfel de sistem. Prin această analiză comparativă se urmărește evidențierea elementelor care pot fi preluate și adaptate în aplicația propusă, precum și a celor care pot fi îmbunătățite pentru a oferi o experiență modernă, intuitivă și performantă.

1. ANALIZA PIETII JOCURILOR INTELECTUALE

1.1. Analiza jocurilor intelectuale.

Scopul acestei secțiuni este de a analiza principalele aplicații client–server existente în domeniul jocurilor de șah online.

Prin această analiză se evidențiază funcționalitățile comune, arhitectura utilizată și punctele forte ale fiecărei soluții, pentru a fundamenta dezvoltarea aplicației proprii [1]. și MEGA, care oferă soluții performante de stocare și partajare a datelor în cloud.

1.1.1. Analiza Aplicației Chess.com

Descriere-generală:

Chess.com este una dintre cele mai populare platforme online de șah, care oferă jocuri în timp real între utilizatori, turnee, lecții interactive și analiză automată a partidelor. Interfața de bază a jocului este reprezentată în figura 1.1

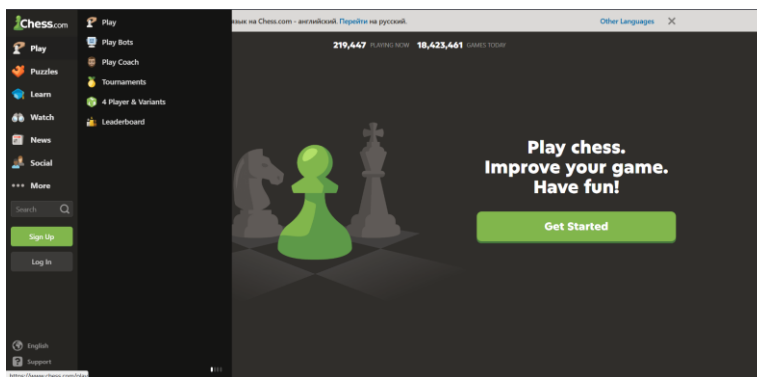


Figura 1.1 – interfața Chess24

Descriere-generală:

Chess.com este una dintre cele mai populare platforme online de șah, care oferă jocuri în timp real între utilizatori, turnee, lecții interactive și analiză automată a partidelor.

Caracteristici principale:

- Joc în timp real între doi jucători prin conexiune client–server.
- Sincronizarea automată a mutărilor între clienți.
- Stocarea partidelor în cloud pentru reluare și analiză.
- Chat integrat și sistem de rating ELO.

Puncte forte: interfață modernă, stabilitate ridicată, funcții avansate de analiză.

Limitări: complexitate mare, necesită conexiune permanentă la internet.

1.1.2. Analiza Aplicației lichess.org

Descriere-generală:

Lichess.org este o platformă open-source gratuită de șah online. Permite jocuri între utilizatori, mod AI, și oferă o comunitate puternică de dezvoltatori.

Caracteristici principale:

- Cod sursă deschis (scris în Scala și TypeScript).
- Posibilitatea de a juca offline sau contra unui bot AI.
- Sincronizare rapidă între client și server.
- Salvarea automată a partidelor în format PGN.

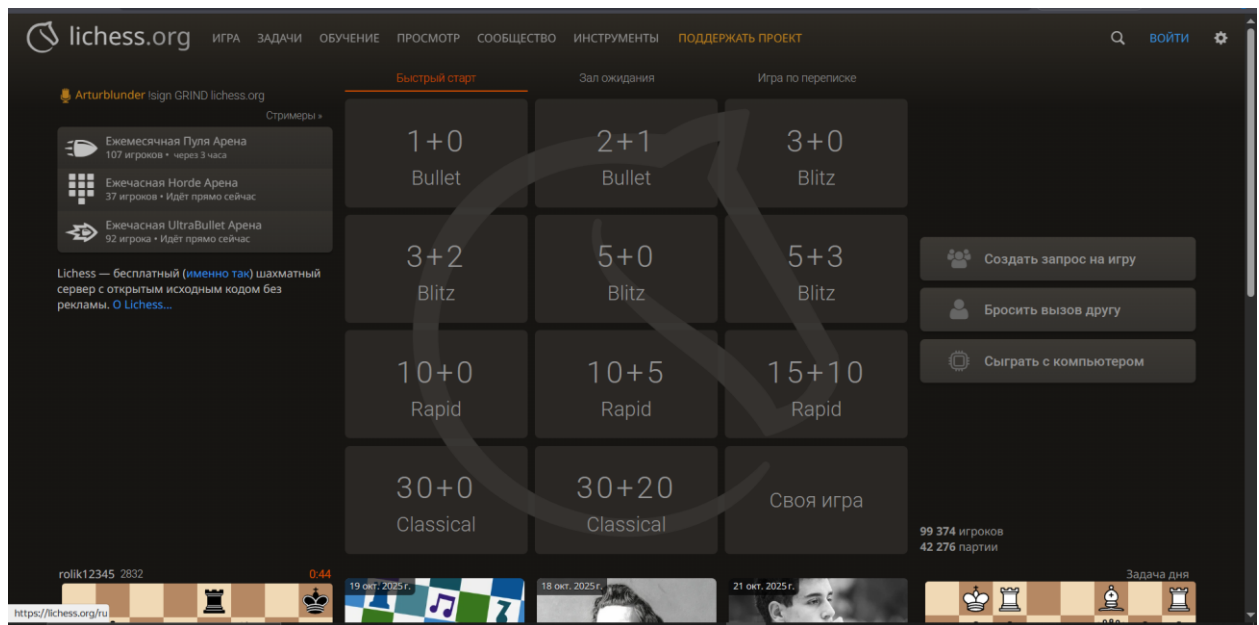


Figura 1.2 – interfața lichess.org

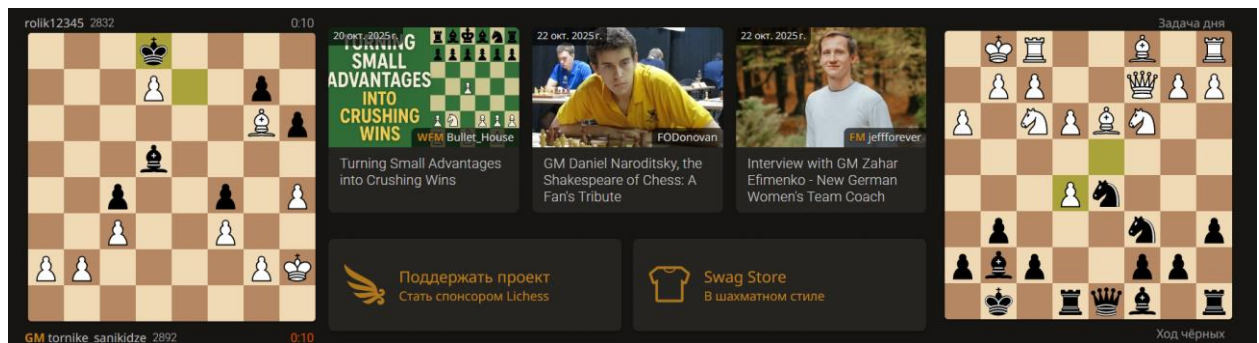


Figura 1.3 – partide in timp real vizualizare

Puncte-forțe: gratuită, transparentă, optimizată pentru performanță.

Limitări: interfață mai simplă decât Chess.com, lipsa unor funcții comerciale.

1.1.3. Analiza Aplicației World Chess

World Chess, cunoscut și ca FIDE Online Arena, este platforma oficială a Federației Internaționale de Șah (FIDE). Permite jucătorilor să participe la meciuri online recunoscute oficial, să obțină rating-uri FIDE online și să câștige titluri oficiale prin competiții digitale.

Caracteristici principale:

- Jocuri online între jucători, cu rating-uri recunoscute oficial.
- Organizarea de turnee și competiții oficiale.
- Posibilitatea de a revizui partidele și de a primi statistici detaliate.
- Arhivarea partidelor și salvarea acestora pentru reluare.



Figura 1.4 – interfața World chess

Avantaje: ideal pentru competiții oficiale și evaluare profesională.

Limitări: accesul complet necesită cont oficial FIDE și unele funcții sunt premium.

1.2. Analiza caracteristicile aplicațiilor

În urma analizei aplicațiilor Chess.com, Lichess și World Chess, se pot evidenția o serie de caracteristici comune, care definesc aplicațiile moderne de șah online:

1. Arhitectură client–server:
Toate aplicațiile se bazează pe un server central care gestionează jocurile și comunicarea între jucători.
2. Sincronizare în timp real:
Mutările efectuate de un jucător sunt transmise imediat către adversar prin conexiune de rețea (WebSocket, TCP/IP).
3. Persistența datelor:
Partidele sunt salvate în baze de date sau fișiere pentru reluare ulterioară.
4. Conturi și autentificare:
Jucătorii au conturi proprii cu istoric de jocuri, clasamente și statistici.
5. Analiză automată (AI):
Majoritatea aplicațiilor oferă posibilitatea de analiză post-partidă prin algoritmi de inteligență artificială.
6. Interfață grafică intuitivă:
Tablele de joc sunt interactive, cu reprezentări vizuale ușor de folosit de către utilizatori.

1.3. Justificarea alegerii caracteristicilor de bază ale aplicației

Pentru aplicația propusă — „Aplicație client–server pentru jocul de șah” — au fost selectate următoarele caracteristici de bază cum este indicat în tabelul 1.1, inspirate din soluțiile analizate anterior:

Tabelul 1.1. Funcționalitatea aplicației

Funcționalitate	Justificare
Conectivitate client–server	Permite jocul între doi utilizatori aflați la distanță, prin trimiterea mutărilor între clienți.
Salvare automată a partidelor	Jucătorii pot relua o partidă întreruptă sau analiza ulterior jocul. Ca referință (Figura 1.2)
Analiză de bază a jocului (AI)	Oferă posibilitatea de a juca împotriva calculatorului sau de a primi sugestii. (Figura 1.3)

Securitatea datelor și autentificare	Fiecare jucător are un cont propriu, pentru protejarea și identificarea sesiunilor. Ca referință (Figura 1.4)
---	---

Prin aceste alegeri, aplicația propusă combină funcțiile esențiale pentru un joc modern de șah online[1], dar rămâne suficient de simplă pentru a fi implementată într-un proiect educațional.

1.4. Caracteristicile de bază aplicației Manager de partide de șah.

Aplicația dezvoltată în cadrul proiectului se bazează pe un model client–server. Principalele componente și funcționalități sunt:

1. Serverul:

- gestionează conexiunile clienților;
- primește și transmite mutările către adversar;
- stochează partidele în fișiere sau baze de date;
- permite reconectarea și reluarea unei partide întrerupte.

2. Clientul:

- oferă interfața grafică pentru tabla de șah; Ca referință este (Fig1.3)
- trimite mutările către server;
- afișează mutările primite de la adversar în timp real; Ca referință este (Fig 1.3)
- permite salvarea și încărcarea partidelor locale.

3. Funcționalități suplimentare (extensibile):

- posibilitatea de a juca împotriva unui bot (AI simplu);
- opțiune de analiză automată a mutărilor;
- partajarea unei partide printr-un cod unic (share link);
- sistem de backup/restaurare a datelor.

2. ANALIZA TEHNOLOGIILOR UTILIZATE ÎN APLICAȚIE

În cadrul dezvoltării aplicației de șah online, au fost utilizate mai multe tehnologii moderne care asigură funcționalitatea completă a sistemului, performanța ridicată și o experiență de utilizare intuitivă.

Aplicația este structurată pe o arhitectură de tip client–server, unde partea de backend (serverul) este realizată în Python, utilizând framework-ul FastAPI, iar partea de frontend (interfața web) este dezvoltată în React.

Comunicarea între cele două părți se face prin API REST și WebSocket, ceea ce permite actualizarea în timp real a mutărilor și a mesajelor de chat dintre jucători.

Pentru stocarea și gestionarea datelor aplicației s-a folosit o bază de date SQLite, ușoară și eficientă, integrată direct în sistem. Interacțiunea cu baza de date este gestionată prin intermediul SQLAlchemy, care permite lucrul cu tabelele prin obiecte Python (modelul ORM(Object-Relational Mapping)).

În ceea ce privește interfața grafică, framework-ul React permite o interacțiune rapidă și fluidă între utilizator și aplicație, oferind o experiență vizuală modernă și adaptabilă. Biblioteca react-chessboard este utilizată pentru afișarea tablei de șah, iar chess.js gestionează regulile de joc și validarea mutărilor.

Astfel, combinația dintre aceste tehnologii — Python (FastAPI), React, SQLite, SQLAlchemy, WebSocket și bibliotecile JavaScript pentru șah — asigură un echilibru optim între performanță, claritate a codului, stabilitate și ușurință în extinderea funcționalităților viitoare ale aplicației.

2.1 Limbajul de programare Python

Aplicația utilizează limbajul de programare Python pentru dezvoltarea părții de backend[2]. Python este un limbaj de nivel înalt, dinamic tipizat, care oferă o sintaxă clară și ușor de înțeles, fiind potrivit atât pentru aplicații web cât și pentru sisteme complexe de procesare a datelor.

În cadrul proiectului, Python a fost folosit pentru implementarea serverului FastAPI, gestionarea rutelor, autentificarea utilizatorilor, lucrul cu baza de date și comunicarea în timp real prin WebSocket.

Avantajele Python includ:

1) suport extins prin biblioteci precum fastapi, sqlalchemy, pydantic, uvicorn;

- 2) dezvoltare rapidă și mentenabilitate ridicată;
- 3) integrare ușoară cu alte servicii și tehnologii.

2.2 Framework-ul FastAPI

FastAPI este un framework modern pentru dezvoltarea de aplicații web și API-uri REST în Python[3]. Este construit peste Starlette (pentru partea de web și WebSocket) și Pydantic (pentru validarea datelor).

În aplicația de șah, FastAPI are rolul principal de server, gestionând:

- 1) rutele pentru înregistrare și autentificare utilizatori;
 - 2) crearea și gestionarea jocurilor (prin WebSocket);
 - 3) interacțiunea cu baza de date SQLite;
- trimiterea și recepționarea mutărilor în timp real între jucători.

Datorită vitezei ridicate și suportului nativ pentru asincronism (async/await), FastAPI asigură performanță și scalabilitate ridicată[4].

2.3 Baza de date SQLite

Pentru stocarea informațiilor aplicației s-a utilizat SQLite, o bază de date relațională ușoară, integrată direct în aplicație, fără a necesita un server separat.

SQLite a fost aleasă pentru:

- 1) simplitate și configurare rapidă;
- 2) performanță bună pentru aplicații de dimensiune medie;
- 3) compatibilitate excelentă cu SQLAlchemy.

În cadrul proiectului, baza de date reține informații precum:

utilizatorii (nume, parolă hash-uită);

- 1) partidele jucate;
- 2) mutările efectuate;
- 3) istoricul de chat între jucători.

2.4. Biblioteca de migrare ORM-ul SQLAlchemy.

Pentru interacțiunea cu baza de date SQLite, aplicația utilizează SQLAlchemy, un ORM (Object Relational Mapper) puternic și flexibil.

Acesta permite lucrul cu tabelele bazei de date prin clase Python, eliminând necesitatea scrierii manuale de interogări SQL.

Avantajele utilizării SQLAlchemy sunt:

- 1) conversia automată între obiecte Python și tabele SQL;
- 2) protecție împotriva injecțiilor SQL;
- 3) portabilitate ușoară către alte baze de date (PostgreSQL, MySQL).

Prin SQLAlchemy s-au definit modelele principale: Player, Game, Move, Chat.

2.5 Framework-ul React

Partea de interfață grafică a fost realizată cu React, o bibliotecă JavaScript dezvoltată de Meta (Facebook) pentru crearea interfețelor dinamice și reactive.

React a permis:

- 1) implementarea unei interfețe moderne, responsive și rapide;
- 2) comunicarea cu backend-ul prin API-uri REST și WebSocket;
- 3) separarea clară a componentelor (login, înregistrare, joc, chat).

În cadrul aplicației, React se ocupă de afișarea tablei de șah (prin biblioteca react-chessboard), gestionarea mutărilor jucătorilor și a mesajelor de chat[6].

2.6 Biblioteca react-chessboard

Pentru randarea tablei de șah în interfața web s-a utilizat biblioteca **react-chessboard**[5], care oferă un set complet de funcționalități pentru mutarea pieselor, rotirea tablei și afișarea pozițiilor.

Aceasta facilitează:

- 1) desenarea automată a tablei conform regulilor standard de șah;
- 2) drag-and-drop pentru mutarea pieselor;

3)actualizarea dinamică a stării în funcție de datele primite de la server. Biblioteca este complet compatibilă cu React și ușor de integrat cu chess.js, folosită pentru logica jocului.

2.7 Biblioteca chess.js

Chess.js este o bibliotecă JavaScript utilizată pentru gestionarea regulilor jocului de șah (mutări valide, detectarea șah-matului, remiză etc.).

Ea rulează pe partea de client și verifică dacă mutările sunt conforme cu regulile jocului.

În combinație cu react-chessboard, aceasta asigură o experiență de joc corectă și interactivă.

2.8. Protocolul WebSocket

Pentru funcționalitatea de joc în timp real și chat între jucători, aplicația folosește protocolul WebSocket, care permite o comunicare bidirecțională între client (React) și server (FastAPI).

Avantajele WebSocket:

- 1)transmiterea rapidă a mutărilor fără reîncărcarea paginii;
- 2)sincronizarea instantanee între jucători;
- 3)suport nativ în FastAPI (websocket endpoints).

Această tehnologie face posibilă implementarea modului multiplayer și a sistemului de chat în aplicație.

2.9 Biblioteca Axios

Pentru comunicarea dintre frontend și backend (în afara conexiunilor WebSocket), a fost folosită biblioteca Axios.

Aceasta simplifică trimiterea cererilor HTTP către server pentru operații precum:

- 1)autentificare;
- 2)înregistrare utilizatori;

3)gestionarea sesiunilor.

Axios oferă o interfață prietenoasă pentru lucrul cu API-urile REST.

3. CREAREA ALGORITMULUI ȘI A PROGRAMULUI

3.1 Descrierea schemei de funcționare a programului

Conform Figurii 3.1, aplicația de șah este realizată pe baza unei arhitecturi client–server, care include patru componente principale: Client, Server, Vizualizare și Baza de date.

Clientul reprezintă interfața prin care utilizatorul interacționează cu aplicația. Acesta trimite cereri către server pentru autentificare, crearea unui joc de șah, conectarea la un joc existent și efectuarea mutărilor. De asemenea, clientul primește datele necesare pentru afișare și le transmite componentei de vizualizare.

Serverul, implementat cu FastAPI, primește cererile de la client și gestionează logica aplicației. La fiecare cerere, serverul deschide o conexiune către baza de date, efectuează operațiile necesare (citire, adăugare, modificare sau ștergere de date), apoi trimite rezultatele înapoi către client. Serverul este responsabil de validarea mutărilor și de sincronizarea jocului între jucători.

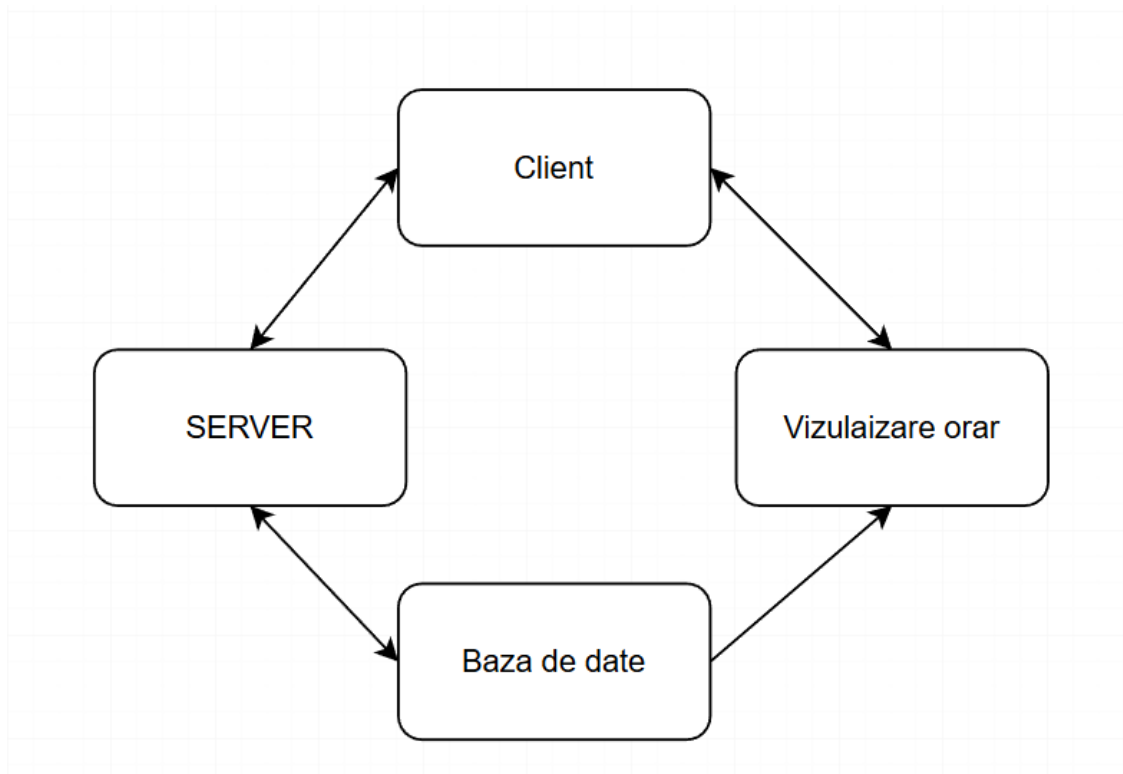


Figura3.1 - Schema de funcționare a programului

Baza de date este utilizată pentru stocarea informațiilor despre utilizatori, jocuri, mutări și mesaje. Aceasta asigură păstrarea istoricului jocurilor și accesul rapid la datele necesare funcționării aplicației.

Componenta de Vizualizare are rolul de a afișa datele primite de la client într-o formă grafică clară și intuitivă. Aceasta prezintă tabla de șah, piesele, mutările efectuate și starea curentă a jocului, permițând utilizatorului să urmărească desfășurarea partidei în timp real.

3.2 Crearea algoritmului programului

Schema prezentată în Figura 3.2 descrie fluxul complet de funcționare al aplicației de șah, evidențiind etapele prin care utilizatorul interacționează cu sistemul și modul în care serverul gestionează logica jocului și datele stocate.

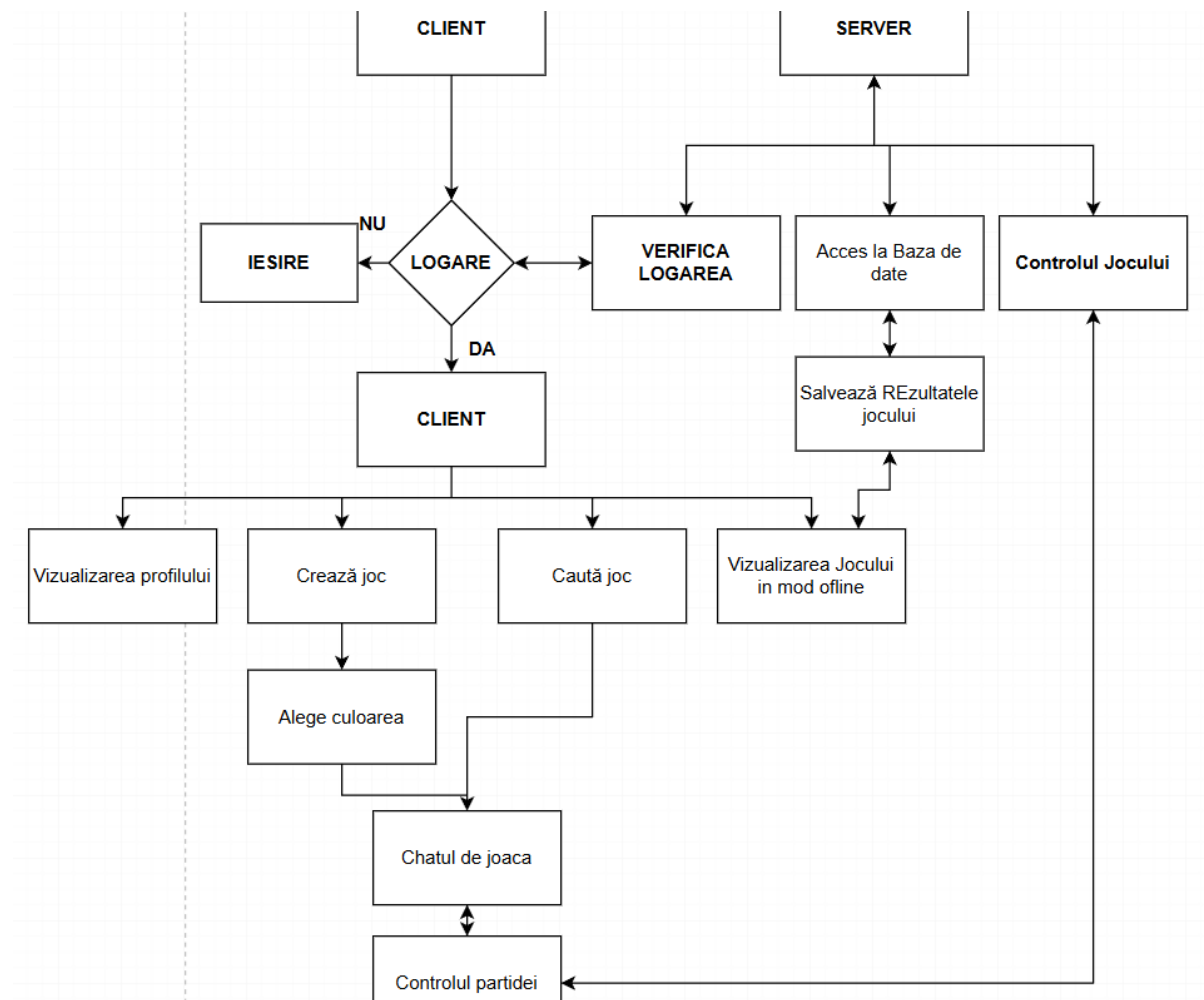


Figura 3.2 - Algoritmul Programului

Procesul începe cu clientul, care inițiază interacțiunea cu aplicația prin accesarea sistemului. Primul pas este etapa de logare, reprezentată în schemă printr-un romb de decizie. În acest punct, aplicația verifică dacă utilizatorul este autentificat.

Dacă logarea nu este validă, utilizatorul este direcționat către opțiunea de ieșire din aplicație. În cazul în care logarea este validă, serverul efectuează verificarea datelor de autentificare, iar utilizatorul obține acces la funcționalitățile aplicației.

În cazul creării unui joc, utilizatorul poate **alege culoarea pieselor**, după care jocul este inițiat. Pe durata partidei, aplicația permite utilizarea **chatului de joc**, precum și **controlul partidei**, inclusiv transmiterea mutărilor către server.

Serverul are rolul de a coordona întregul proces al jocului. Acesta verifică logarea, gestionează regulile jocului de șah, controlează desfășurarea partidei și comunică permanent cu **baza de date**. Toate informațiile importante, precum mutările efectuate și rezultatul final al jocului, sunt salvate în baza de date pentru a permite păstrarea istoricului.

Baza de date este utilizată pentru stocarea informațiilor despre utilizatori, jocuri și rezultate. Aceasta asigură consistența și integritatea datelor și permite reluarea sau analizarea jocurilor ulterior.

Prin acest algoritm, aplicația de șah oferă un flux coerent și eficient, asigurând comunicarea corectă între client, server și baza de date, precum și o experiență de joc stabilă și sincronizată pentru utilizatori.

3.3 Descrierea programului

Aplicația dezvoltată este un sistem client–server pentru jocul de șah online, conceput pentru a permite utilizatorilor să joace partide de șah în timp real, să comunice între ei și să păstreze istoricul jocurilor. Aplicația este structurată în două module principale: serverul și clientul, fiecare având un rol bine definit în funcționarea întregului sistem.

Serverul reprezintă componenta centrală a aplicației și implementează logica principală a jocului. Acesta este dezvoltat utilizând framework-ul FastAPI și este responsabil de gestionarea autentificării utilizatorilor, crearea și administrarea jocurilor de șah, validarea mutărilor conform regulilor jocului și sincronizarea partidelor între jucători. Serverul primește cererile de la client, le procesează și interacționează cu baza de date pentru a salva sau a furniza informațiile necesare.

Pentru desfășurarea jocului în timp real, serverul utilizează conexiuni WebSocket, prin intermediul cărora sunt transmise mutările jucătorilor, mesajele din chat și actualizările stării jocului[7]. Acest mecanism asigură o comunicare rapidă și sincronizată între toți utilizatorii conectați la aceeași partidă.

Clientul este destinat utilizatorilor finali și oferă interfața prin care aceștia interacționează cu aplicația. Prin intermediul clientului, utilizatorii pot să se autentifice, să își vizualizeze profilul, să creeze jocuri noi, să caute jocuri existente și să participe la partide de șah. Interfața permite afișarea tablei de șah, realizarea mutărilor, utilizarea chatului de joc și urmărirea evoluției partidei.

Clientul este proiectat să fie intuitiv și ușor de utilizat, concentrându-se pe experiența de joc. În plus, aplicația oferă posibilitatea vizualizării jocului în mod offline, utilizând datele salvate local, astfel încât utilizatorul să poată analiza partidele chiar și în lipsa unei conexiuni active la internet.

Baza de date, implementată folosind SQLite, este utilizată pentru stocarea informațiilor despre utilizatori, jocuri, mutări și mesaje[10]. Aceasta este structurată relațional, permițând păstrarea istoricului complet al fiecărei partide și asigurând integritatea datelor. Orice modificare realizată prin intermediul serverului devine imediat disponibilă pentru clienții conectați.

Aplicația este dezvoltată utilizând tehnologii moderne precum Python, FastAPI, SQLite, React și Next.js, ceea ce asigură o performanță bună și o arhitectură scalabilă. Comunicarea dintre client și server este realizată printr-un API RESTful pentru operațiunile standard și prin WebSocket-uri pentru funcționalitățile în timp real.

Securitatea aplicației este asigurată printr-un sistem de autentificare bazat pe JSON Web Tokens (JWT), iar parolele utilizatorilor sunt stocate în formă criptată, utilizând algoritmul Argon2. Această arhitectură client–server permite o separare clară a responsabilităților, oferind un nivel ridicat de securitate, modularitate și posibilitatea extinderii aplicației cu funcționalități noi în viitor.

3.3.1 Partea Client

Modulul client al aplicației este destinat utilizatorilor finali și oferă o interfață intuitivă pentru jocul de șah online. Aplicația client este dezvoltată utilizând framework-ul React, asigurând o experiență modernă, responsivă și ușor de utilizat pe diferite dispozitive.

Interfața principală permite utilizatorilor să se autentifice, să își vizualizeze profilul, să creeze jocuri noi de șah sau să caute jocuri existente. După conectarea la o partidă, utilizatorul are acces la tabla de șah, afișată cu ajutorul bibliotecii react-chessboard, care oferă o reprezentare vizuală clară și interactivă a jocului.

Logica jocului de șah, inclusiv validarea mutărilor, detectarea stărilor speciale (șah, șah mat, remiză) și gestionarea regulilor jocului, este realizată cu ajutorul bibliotecii chess.js. Aceasta asigură corectitudinea mutărilor efectuate de jucători înainte ca acestea să fie transmise către server.

Aplicația implementează un mecanism de stocare locală bazat pe localStorage, care permite salvarea informațiilor despre utilizator și a datelor jocului. Acest mecanism oferă posibilitatea vizualizării jocului în mod offline, permițând utilizatorului să analizeze partidele salvate chiar și în lipsa unei conexiuni active la internet.

Comunicarea în timp real între jucători este realizată prin intermediul WebSocket-urilor[7]. Clientul stabilește automat o conexiune WebSocket cu serverul la inițierea jocului și primește instantaneu mutările adversarului, mesajele din chat și actualizările stării partidei. La primirea unui eveniment nou, interfața este actualizată imediat, fără a fi necesară reîncărcarea paginii.

În cazul pierderii conexiunii WebSocket, aplicația implementează un mecanism de reconectare automată. Dacă reconectarea nu este posibilă, clientul utilizează cereri HTTP către server pentru a menține sincronizarea jocului.

Clientul oferă funcționalități precum:

- crearea și căutarea jocurilor de șah;
- alegerea culorii pieselor la inițierea unei partide;
- utilizarea chatului de joc pentru comunicarea dintre jucători;
- controlul partidei și afișarea rezultatului final.

Comunicarea cu serverul pentru operațiile standard este realizată prin intermediul bibliotecii Axios, utilizată pentru trimiterea cererilor HTTP[9]. Cererile sunt structurate printr-un API bine definit, asigurând un schimb de date eficient între client și server.

Sistemul de autentificare utilizează JSON Web Tokens (JWT), token-ul fiind stocat local și atașat automat la fiecare cerere către server. În cazul în care token-ul este invalid sau expirat, utilizatorul este redirecționat automat către pagina de autentificare.

Arhitectura modulului client este organizată în componente reutilizabile și servicii dedicate, facilitând mentenanța și extinderea aplicației. Interfața este proiectată într-un stil clar și minimalist, punând accent pe experiența de joc și pe ușurința în utilizare, oferind o interacțiune stabilă și plăcută pentru utilizatori.

3.3.2 Partea Server

Modulul server reprezintă componenta centrală a aplicației de șah și este responsabil de gestionarea logicii jocului, a autentificării utilizatorilor și a comunicării dintre clienți. Serverul este dezvoltat utilizând framework-ul FastAPI, care permite construirea unui API performant și ușor de extins.

Serverul primește cereri de la client prin intermediul API-ului REST, acestea fiind utilizate pentru operații precum autentificarea utilizatorilor, crearea jocurilor de șah, căutarea jocurilor disponibile și obținerea informațiilor despre jocuri. Cererile sunt procesate, validate și tratate conform regulilor aplicației, iar răspunsurile sunt returnate către client în format JSON.

Pentru gestionarea datelor persistente, serverul utilizează o bază de date SQLite[10], accesată prin intermediul ORM-ului SQLAlchemy[11]. SQLAlchemy permite maparea tabelelor bazei de date în modele obiectuale, facilitând operațiile de creare, citire, actualizare și ștergere a datelor. În baza de date sunt stocate informații despre utilizatori, jocuri, mutări și mesaje de chat, asigurând păstrarea istoricului fiecărei partide.

Logica jocului este coordonată de server, care gestionează starea fiecărei partide, ordinea mutărilor și sincronizarea dintre jucători. Serverul primește mutările de la client, le validează din punct de vedere al structurii și al apartenenței la jocul corespunzător, apoi le transmite celui alt jucător.

Pentru funcționalitățile în timp real, serverul implementează conexiuni WebSocket[7]. Prin intermediul acestora, mutările, mesajele din chat și actualizările stării jocului sunt transmise instantaneu tuturor clienților conectați la aceeași partidă. Acest mecanism permite desfășurarea jocului fără întârzieri și fără a fi necesare cereri repetate către server.

Serverul gestionează autentificarea utilizatorilor utilizând JSON Web Tokens (JWT). La autentificare, serverul generează un token care este transmis clientului și utilizat ulterior pentru autorizarea cererilor. La fiecare cerere protejată, serverul verifică validitatea token-ului, asigurând accesul doar utilizatorilor autentificați.

Arhitectura serverului este organizată modular, separând clar rutele API, logica aplicației și modelele bazei de date. Această structură contribuie la creșterea clarității codului, la ușurința mentenanței și la posibilitatea extinderii aplicației cu funcționalități noi.

Prin utilizarea FastAPI, SQLAlchemy, SQLite și WebSocket, modulul server oferă o soluție eficientă, sigură și scalabilă pentru gestionarea aplicației de șah, asigurând o comunicare stabilă și sincronizată cu partea de client.

3.3.3 Baze de Date

Baza de date a aplicației de șah este implementată folosind SQLite, o bază de date relațională ușoară[10], gestionată prin intermediul ORM-ului SQLAlchemy[11]. Această soluție oferă o bună performanță pentru aplicații de dimensiune mică și medie, precum și o portabilitate ridicată a datelor, fiind potrivită pentru dezvoltare și testare locală.

Structura bazei de date este organizată în mai multe tabele principale, care modelează entitățile esențiale ale aplicației de șah.

Tabelul users stochează informații despre utilizatorii aplicației. Acesta conține date precum identificatorul unic al utilizatorului, numele de utilizator, hash-ul parolei și data creării contului. Parolele sunt stocate în formă criptată, asigurând securitatea informațiilor de autentificare.

Tabelul games reprezintă nucleul aplicației și conține informații despre jocurile de șah create. Acesta include identificatorul jocului, jucătorii participanți, starea jocului (în așteptare, în desfășurare sau finalizat), rezultatul final și momentele de început și de sfârșit ale partidei.

Tabelul moves este utilizat pentru stocarea istoricului mutărilor efectuate într-un joc. Fiecare înregistrare conține referința către jocul corespunzător, jucătorul care a efectuat mutarea, poziția inițială, poziția finală și momentul realizării mutării. Această structură permite reconstruirea completă a partidei și analizarea jocurilor ulterior.

Tabelul chats stochează mesajele schimbate între jucători în timpul unei partide. Acesta conține referințe către joc și utilizator, mesajul transmis și data trimiterii, permițând păstrarea istoricului comunicării din cadrul jocului.

Relațiile dintre tabele sunt definite prin chei externe, asigurând integritatea referențială a datelor. De exemplu, fiecare mutare din tabelul moves este asociată unui joc valid din tabelul games și unui utilizator existent din tabelul users. În mod similar, mesajele din tabelul chats sunt legate de jocurile și utilizatorii corespunzători.

Gestionarea relațiilor și a operațiilor asupra bazei de date este realizată prin SQLAlchemy, care permite efectuarea operațiilor de creare, citire, actualizare și ștergere într-un mod sigur și eficient. Modificările realizate asupra datelor sunt reflectate imediat în aplicație, asigurând consistența informațiilor între server și clienții conectați.

Prin această structură relațională, baza de date oferă suport pentru desfășurarea jocurilor de șah în timp real, păstrarea istoricului partidelor și gestionarea eficientă a utilizatorilor, contribuind la stabilitatea și funcționalitatea aplicației.

CONCLUZIE

În cadrul acestui proiect a fost realizată o aplicație client–server de șah, care permite desfășurarea jocurilor în timp real între doi utilizatori, folosind tehnologii moderne și o arhitectură bine structurată. Scopul principal al aplicației a fost implementarea unui sistem stabil și eficient, capabil să gestioneze comunicarea dintre client și server, sincronizarea mutărilor și stocarea datelor jocului.

Aplicația este construită pe o arhitectură client–server, în care partea de client asigură interfața utilizatorului și experiența de joc, iar partea de server gestionează logica aplicației, regulile jocului și comunicarea cu baza de date. Utilizarea WebSocket-urilor permite transmiterea în timp real a mutărilor și a mesajelor, oferind o experiență de joc fluentă și sincronizată.

Folosirea tehnologiilor precum FastAPI, React, SQLite și SQLAlchemy a contribuit la realizarea unei aplicații performante, ușor de întreținut și extensibile. Bibliotecile react-chessboard și chess.js au asigurat afișarea corectă a tablei de șah și validarea mutărilor conform regulilor jocului, iar Axios a facilitat comunicarea eficientă dintre client și server.

Aplicația permite autentificarea utilizatorilor, crearea și gestionarea jocurilor, comunicarea prin chat și păstrarea istoricului partidelor, oferind astfel funcționalitățile de bază necesare unui sistem de șah online. Structura modulară a aplicației permite extinderea acesteia în viitor, prin adăugarea unor funcționalități suplimentare, precum jocul împotriva inteligenței artificiale, sistem de rating sau turnee online.

În concluzie, proiectul realizat demonstrează aplicarea practică a conceptelor de programare client–server și comunicare în timp real, oferind o soluție funcțională și bine organizată pentru jocul de șah online.

BIBLIOGRAFIE

Chess design:

1) Srivastava, A. K. (2025). Design an Online Chess Service – System Requirements. Medium blog post. Provides requirements and architectural considerations for online chess service in a client–server paradigm.

Python:

2) Lubanović, Bill. *FastAPI: Modern Python Web Development*. O'Reilly Media, 2023.

FastAPI:

3) “FastAPI – Introduction.” *GeeksforGeeks*, 23 sept. 2025.

4) “FastAPI.” *FastAPI Documentation*. tiangolo.com.

React:

5) “react-chessboard.” *npm package*. 29 oct. 2025.

6) “How to build a multiplayer chess game with React.” Pachi, SuperViz Blog, 26 sept. 2024.

7) MDN Web Docs. *WebSocket API*.

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

8) *chess.js – JavaScript chess library*. GitHub Repository.

<https://github.com/jhlywa/chess.js>

9) *Axios Documentation*.

<https://axios-http.com/docs/intro>

10) *SQLite Documentation*.

<https://www.sqlite.org/docs.html>

11) *SQLAlchemy Documentation*.

<https://docs.sqlalchemy.org/>